

Clocktree Synthesis and Timing Closure

Louis Carlin

May 2025

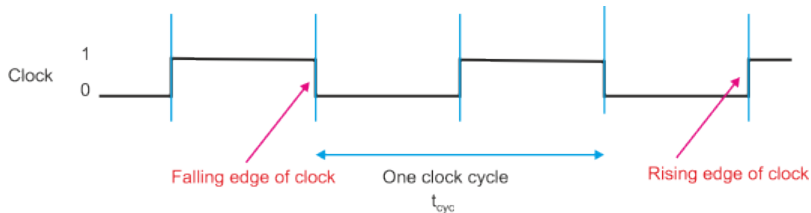
Overview

- ▶ Chips run in *cycles*
- ▶ A typical modern chip might operate at 3.4GHz (1 cycle every 294 picoseconds).
- ▶ **Clocktree Synthesis:** We need to distribute clock signals to synchronize operations.
- ▶ **Timing Closure:** we need to ensure the worst case signal propagation time does not exceed the cycle time.



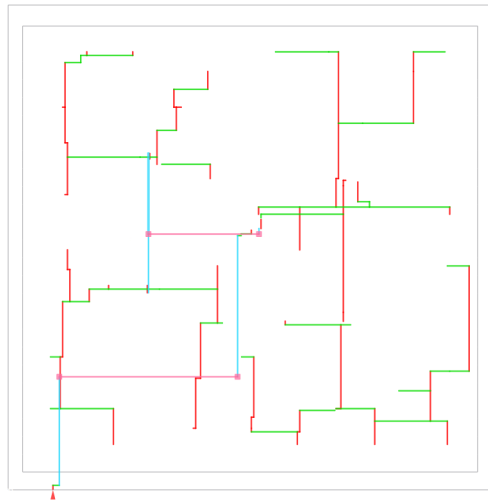
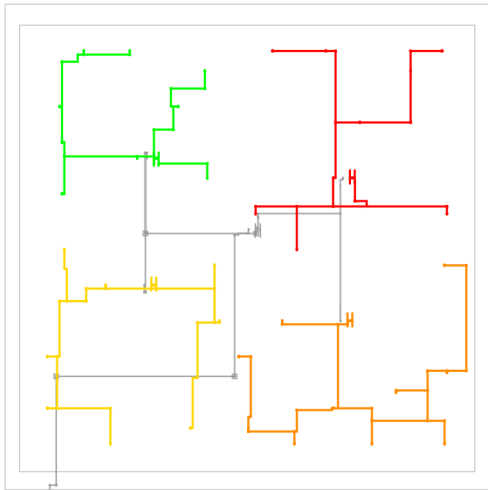
Clock Sources and Sinks

- ▶ A clock source (e.g. an electronic oscillator) produces a periodic signal, such as a square wave.



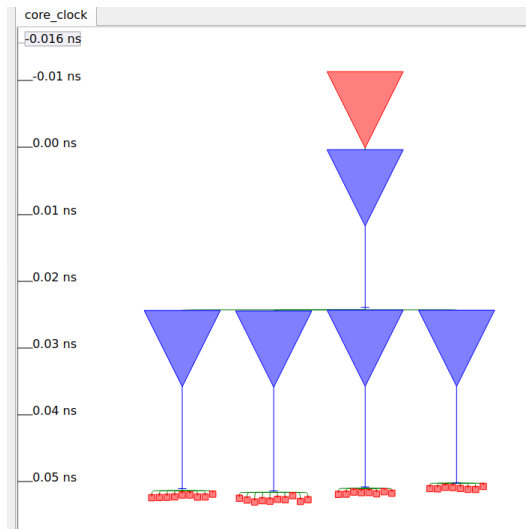
- ▶ A network of wires and clock buffers propagates this signal to the *clock sinks* (e.g. latch clock pins).
- ▶ We need to design this network.

Clock Trees Example



Clock Tree Synthesis: Objectives

- ▶ **Minimal Skew:** signals should arrive at clock sinks at a similar time.
- ▶ **Minimal Latency:** the maximum time for the signal to travel from the clock source to a clock sink should not be too large.
- ▶ **Tertiary Objectives:** wirelength, power consumption, space usage, etc.

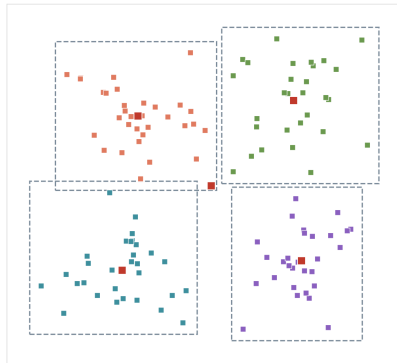
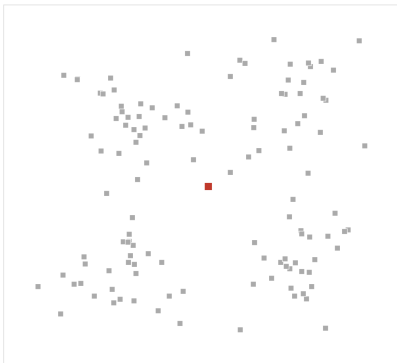


Clock Tree Synthesis in OpenROAD

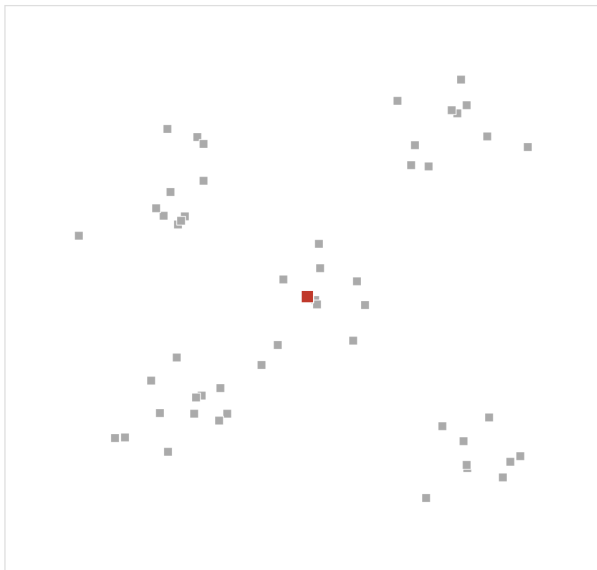
- ▶ CTS is available in OpenROAD via the command `clock_tree_synthesis`.
- ▶ Preclustering → H-tree Generation → Buffering

Preclustering

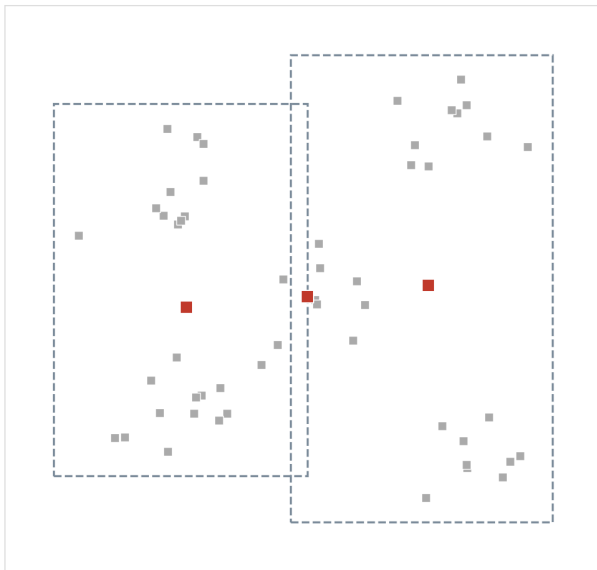
- Preclustering is enabled* with the flag `-sink_clustering_enable`



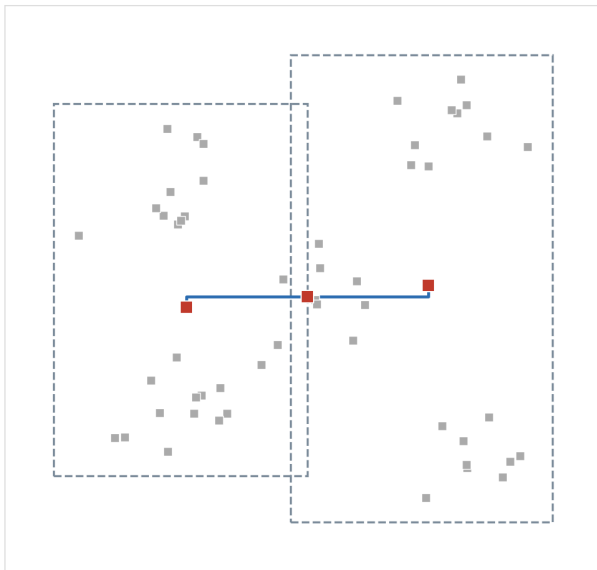
H-tree Generation



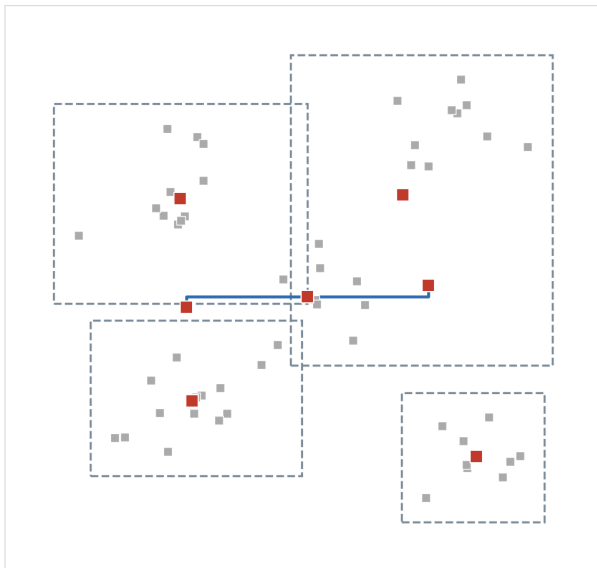
H-tree Generation



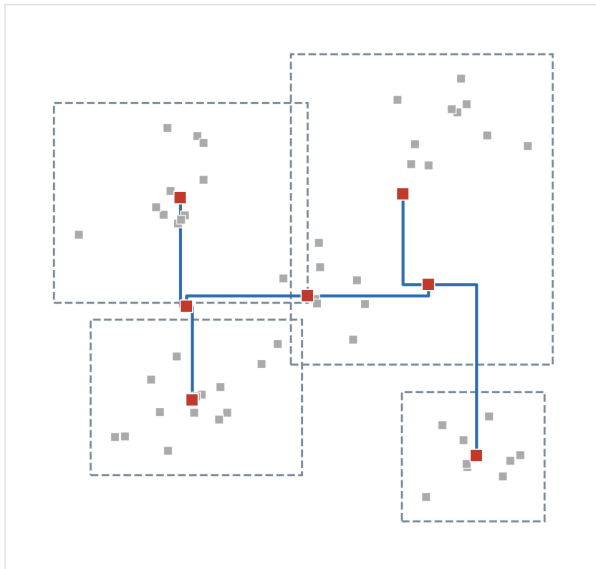
H-tree Generation



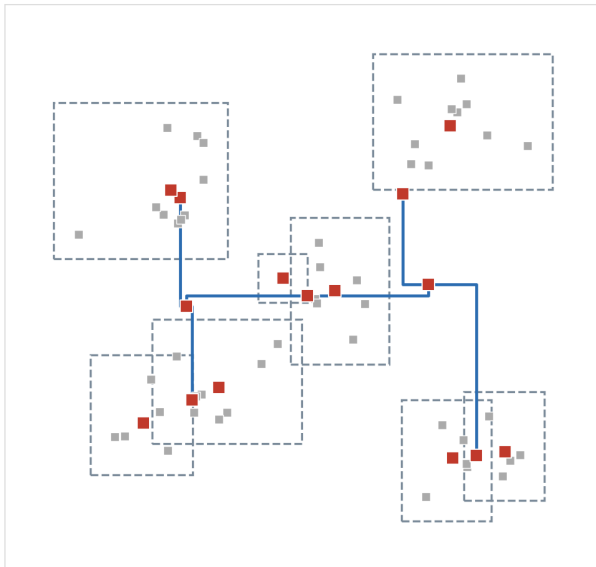
H-tree Generation



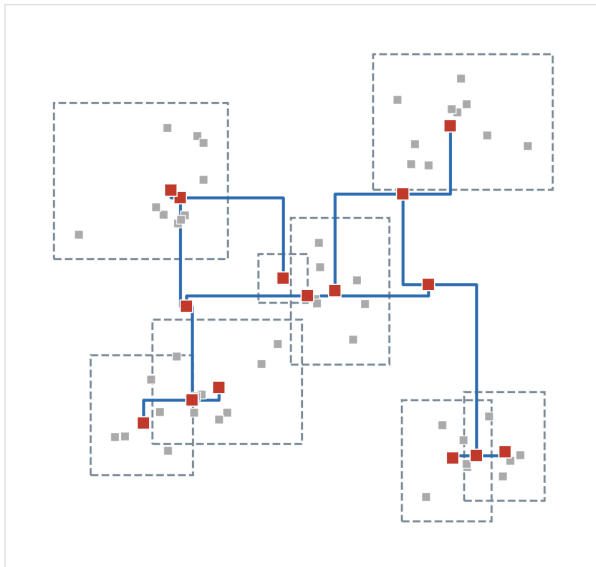
H-tree Generation



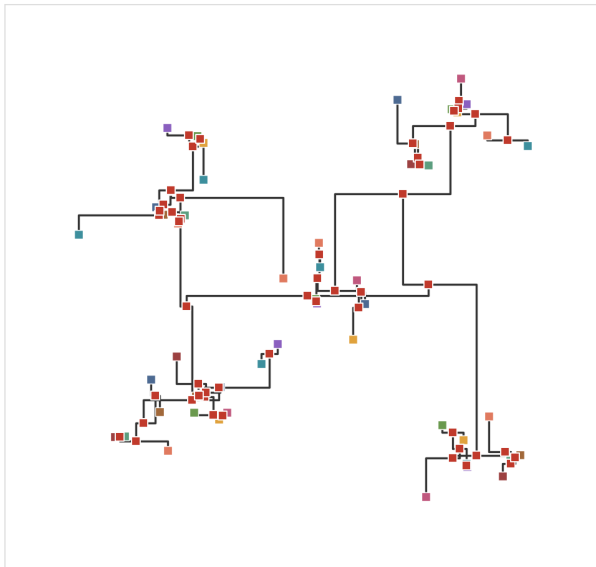
H-tree Generation



H-tree Generation



H-tree Generation



Constrained K-Means Clustering

1. Start with some initial cluster centres $(X_1, Y_1), \dots, (X_K, Y_K)$.
2. Assign sinks to closest clusters such that the clusters are roughly balanced.
3. Update the centre of each cluster to the mean of its assigned sink positions.
4. If not converged, go to Step 2.

Constrained K-Means Clustering

$$\min_{\{z_{ij}\}} \sum_{i=1}^N \sum_{j=1}^K d_{ij}^p z_{ij}$$

$$\text{s.t.} \quad \sum_{j=1}^K z_{ij} = 1, \quad \forall i = 1, \dots, N,$$

$$\sum_{i=1}^N z_{ij} \leq C_j, \quad \forall j = 1, \dots, K,$$

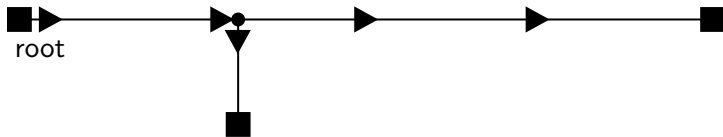
$$z_{ij} \in \{0, 1\}, \quad \forall i, j.$$

where

$$d_{ij} = |x_i - X_j| + |y_i - Y_j|.$$

Buffering

- ▶ Precompute lookup tables for various wirelengths ($15\mu\text{m}$, $30\mu\text{m}$, $45\mu\text{m}$, $60\mu\text{m}$, $75\mu\text{m}$, $90\mu\text{m}$)
- ▶ Enumerate all possible buffering solutions with the minimum granularity of $15\mu\text{m}$
- ▶ Each solution is evaluated with various input slews and downstream capacitances.
- ▶ Solutions are pruned until only a few solutions remain for each tuple (length, input slew, downcap).
- ▶ To buffer longer wires we run a dynamic program which uses the precomputed solutions for small lengths.



Timing Closure

- ▶ We need to ensure that signals do not arrive too late (setup violation) or too early (hold violation).
- ▶ OpenROAD has the command `repair_timing` which does both.
- ▶ **Slack:** the difference between required arrival time and arrival time.
- ▶ As technologies get smaller, a general trend is that timing needs to be considered in earlier steps (placement, routing) rather than fixed in a post-optimization.

 Mateus Fogaça, Eder Monteiro, Marcelo Danigno, Isadora Oliveira, Paulo F Butzen, and Ricardo Reis.

Contributions to openroad from abroad: experiences and learnings.

In Proceedings of the 39th International Conference on Computer-Aided Design,
pages 1–8, 2020.

 Kwangsoo Han, Andrew B Kahng, and Jiajia Li.

Optimal generalized h-tree topology and buffering for high-performance and low-power clock distribution.

IEEE transactions on computer-aided design of integrated circuits and systems,
39(2):478–491, 2018.